



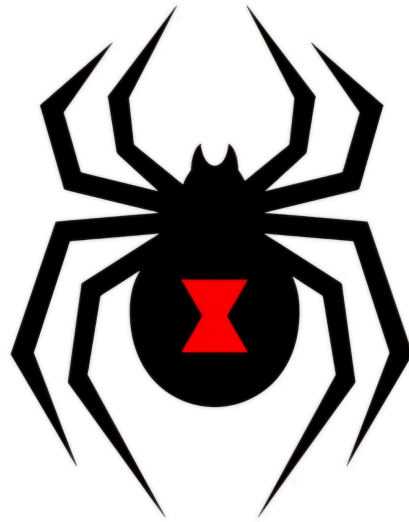
UNSA

Universidad Nacional de San Agustín

“AÑO DE LA ESPERANZA Y EL FORTALECIMIENTO DE LA DEMOCRACIA”

FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



VIUDA NEGRA

Informe de pruebas de integración

ASIGNATURA:

Pruebas de Software

DOCENTE:

Ing. Robert Edison Arisaca Mamani

INTEGRANTES:

Tejada Lazo Jordy Rolando (Test Lead)

Hurtado Bejarano Michael Steve (Test Analyst)

Jaita Chura Jose Manuel (Test Design)

Yare Chulunquia Kevin Pedro (Test Analyst)

Del Castillo Montoya Christopher Brad (Architect)

INVENTREE Plan de Pruebas de Integración	UNSA - EPIS Página 2
---	---------------------------------------

Historial de versiones

Versión	Autor(es)	Descripción	Fecha
1.0	Jaita Chura Jose Manuel	Versión inicial del documento	07/07/2026
2.0	Hurtado Bejarano Michael Steve	Complementación del documento	08/07/2026

INVENTREE Plan de Pruebas de Integración	UNSA - EPIS Página 3
---	---------------------------------------

1. Introducción.....	4
1.1. Propósito.....	4
1.2. Alcance.....	4
1.3. Referencias.....	4
2. Entorno de pruebas.....	5
2.1. Configuración.....	5
2.2. Limitaciones.....	6
3. Resultados de ejecución.....	8
3.1. Detalles de ejecución.....	8
3.2.1. Solicitud de una cuenta de instructor.....	8
4. Cobertura.....	8
5. Métricas de Prueba Consolidadas.....	12
6. Análisis General.....	13
6.1. Consistencia entre módulos.....	13
6.2. Robustez general del sistema.....	14
6.3. Relación entre lo planificado.....	14
6.4. Validez cruzada entre CI y ejecución local.....	15
7. Conclusiones y Recomendaciones.....	15
7.1. Conclusiones.....	15
7.2. Recomendaciones.....	15

INVENTREE Plan de Pruebas de Integración	UNSA - EPIS Página 4
---	---

1. Introducción

1.1. Propósito

El propósito de este informe es documentar y consolidar los resultados obtenidos tras la re-ejecución de la suite de pruebas de integración del sistema InvenTree v1.3.0, evaluando la calidad técnica del software bajo un enfoque metodológico Top-Down guiado por el pipeline CI/CD. El documento tiene como fin certificar que las interacciones entre las distintas capas del sistema —específicamente el backend Django, la API REST, la base de datos de pruebas (SQLite) y las de producción (PostgreSQL, MySQL), el sistema de caché Redis y el ecosistema de plugins— funcionan de manera correcta, coordinada y coherente. Con esto, se busca validar el cumplimiento de los criterios de aceptación académicos, verificar que la cobertura de código del backend supere el umbral mínimo del 85%

1.2. Alcance

El alcance de este informe se limita exclusivamente a la verificación de la capa de integración de los componentes del núcleo de InvenTree y sus flujos automatizados en el repositorio fork de GitHub Actions, recopilando los resultados de más de 452 métodos de prueba distribuidos en los módulos de Part, Stock, Order, Build, Company, Plugin, Users y Common. Incluye la validación de la compatibilidad multiplataforma de las migraciones de bases de datos con PostgreSQL 17 y MySQL 9, el comportamiento del cliente oficial inventree-python contra el servidor de pruebas y la ejecución de flujos End-to-End en la interfaz gráfica del frontend React. Quedando fuera la corrección de defectos dentro del código fuente de terceros, las auditorías de seguridad avanzada o penetración, las pruebas de rendimiento, carga y estrés bajo escenarios de alta concurrencia.

1.3. Referencias

- **ISO/IEC/IEEE 29119:** Estándar internacional para pruebas de software.
- Documentación oficial del sistema TEAMMATES

INVENTREE Plan de Pruebas de Integración	UNSA - EPIS Página 5
---	---------------------------------------

- Repositorio oficial de InvenTree:
<https://github.com/inventree/InvenTree.git>
-

2. Entorno de pruebas

2.1. Configuración

- **Configuración del Entorno:**

Para garantizar la reproducibilidad y la consistencia de los resultados, las pruebas de integración se estructuraron bajo una estrategia híbrida que combina la potencia de la automatización en la nube con la flexibilidad de un entorno local de respaldo.

- **Sub-entorno de Integración Continua (CI):**

El ecosistema principal de ejecución se desplegó en GitHub Actions a través del fork del repositorio oficial. Este entorno se encuentra orquestado por tres flujos de trabajo clave analizados en las directrices técnicas:

- qc_checks.yaml: Encargado de validar de forma jerárquica el análisis estático de código, la consistencia del esquema OpenAPI (schema), y la integración de la API REST backend contra múltiples motores de bases de datos utilizando los subprocesos de aislamiento analizados.
- frontend.yaml: Responsable de compilar la SPA en React y coordinar la ejecución paralela mediante shards de las pruebas funcionales End-to-End.
- import_export.yaml: Flujo especializado que valida la consistencia referencial y la transferencia de registros en caliente simulando un ciclo completo entre PostgreSQL y SQLite.

- **Sub-entorno Local:**

Como mecanismo de soporte, contingencia y diagnóstico de fallos rápidos, se configuró una estación de trabajo bajo la siguiente arquitectura:

INVENTREE Plan de Pruebas de Integración	UNSA - EPIS Página 6
---	---

- **Sistema Operativo: WSL2 (Windows Subsystem for Linux)** ejecutando una distribución nativa de Ubuntu 22.04 LTS para garantizar paridad de comportamiento con los runners Linux de GitHub.
- **Capa de Servicios y Persistencia:** Contenedores aislados mediante Docker ejecutando instancias reales de PostgreSQL 17 (motor de persistencia de producción) y Redis 8 (actuando como bróker de mensajería asíncrona para Django Q2 y caché del sistema)
- **Entorno de Ejecución Backend:** Python 3.12 administrado mediante entornos virtuales nativos (venv) para el aislamiento de las dependencias e invocación de tareas mediante el task runner del proyecto (tasks.py)

2.2. Limitaciones

- Debido a que el repositorio opera sobre un fork estudiantil público que carece de secretos de organización pre-configurados (como el CODECOV_TOKEN), la suite automatizada experimenta restricciones para la subida directa de métricas hacia la plataforma web. Esta limitación se mitiga mediante la extracción e inspección directa de los reportes en formato XML (coverage xml) generados localmente o descargados como artefactos de ejecución del CI.
- Los recursos informáticos asignados a GitHub Actions poseen restricciones en potencia de cómputo por instancia. La alta demanda que exige paralelizar la suite de pruebas del frontend React introduce riesgos de timeouts involuntarios por falta de recursos de hardware en el servidor asignado.
- Las pruebas End-to-End basadas en Playwright en el flujo de interfaz gráfica tienden a presentar comportamientos inestables o flaky debido a sutiles latencias en el renderizado de los componentes de Mantine UI, la carga de datos demo asíncronos o desfases en la red interna del runner, lo que puede provocar falsos negativos sin que exista un defecto real en el código fuente.

INVENTREE Plan de Pruebas de Integración	UNSA - EPIS Página 7
---	---------------------------------------

2.3. Estrategia de Ejecución

La metodología adoptada para este ciclo responde estrictamente a un enfoque de Integración Incremental Top-Down guiada y automatizada por el pipeline CI/CD oficial.

En lugar de diseñar nuevos casos de prueba ad-hoc desde cero, la estrategia del equipo se concentró de manera exhaustiva en re-ejecutar, estudiar y documentar los 452+ métodos de prueba de integración oficiales distribuidos en los archivos `test_api.py` de la plataforma

3. Resultados de ejecución

3.1. Resumen General

Workflow	Ejecución	Resultado	Duración	Artifacts
QC (qc_checks.yaml)	QC #3	PASSED	1h 5m 25s	4
Import / Export	Import/Export #3	PASSED	7m 56s	—
Frontend (Playwright E2E)	Frontend #2	FAILED (parcial)	23m 36s	6

De los 3 workflows evaluados, 2 finalizaron sin fallos y 1 (Frontend) presentó un defecto reproducible

3.2. Detalle por job — QC #3

Desglose de los 15 jobs que componen el workflow `qc_checks.yaml` en esta ejecución:

Job	Resultado	Duración	Corresponde a (Plan, Sec. 5.3)
Filter (paths-filter)	PASS	10s	Detección de cambios
Style [prek]	PASS	57s	code-style
Style [Typecheck]	PASS	2m 44s	typecheck
Tests - API Schema Documentation	PASS	3m 18s	schema
Style [Documentation]	PASS	32s	Documentación
Tests - Migrations [PostgreSQL]	PASS	1h 5m	INT-032

INVENTREE Plan de Pruebas de Integración	UNSA - EPIS Página 8
---	---------------------------------------

Tests - Full Migration [SQLite]	PASS	4m 47s	INT-028 a INT-031
Tests - inventree-python	PASS	5m 24s	INT-038, INT-039
Tests - DB [SQLite] + Coverage 3.12/3.14	PASS	(paralelo)	INT-001 a INT-027 (coverage)
Tests - DB [PostgreSQL]	PASS	26m 26s	INT-010, INT-016, INT-018, INT-022 a INT-024
Tests - DB [MySQL]	PASS	21m 48s	Compatibilidad cross-DB
Security [Zizmor]	PASS	17s	Análisis de seguridad de workflows
Tests - Performance	SKIPPED	—	INT-040 (solo repo oficial, no aplica al fork)
Push new schema	SKIPPED	—	Solo si hay diffs de schema a publicar

Como resultado final, 13 de 13 jobs no omitidos finalizaron exitosamente (100%). Los 2 jobs restantes fueron omitidos por diseño del propio pipeline (no por fallo), consistente con lo previsto en el Plan para INT-040.

3.3. Defectos encontrados

DEF-01 — Falla reproducible en Company - Supplier Parts

Campo	Detalle
ID Defecto	DEF-01
Frontera/Componente afectado	Frontend (React) ↔ API Backend — vista Company → Supplier Parts
Severidad	Media (afecta un flujo de UI específico, no bloquea el sistema)
Descripción	El test E2E espera que la interfaz muestre un contador de paginación (patrón "1 - 25 / 77X") tras limpiar filtros de tabla, pero el elemento nunca se vuelve visible dentro del timeout de 90s
Reproducibilidad	Falla de forma consistente en 2 navegadores distintos: Chromium y Firefox
Comportamiento esperado	El contador de resultados se actualiza y se vuelve visible tras limpiar los filtros
Comportamiento real	Timeout de 90s esperando el texto del contador
Causa raíz (hipótesis)	Posible condición de carrera entre la limpieza de filtros y la actualización asíncrona de la tabla (React Query)
Registro formal	No se registró como GitHub Issue dedicado (decisión de equipo); queda documentado en este informe como hallazgo formal de integración
Estado	Documentado, pendiente de investigación de causa raíz

INVENTREE Plan de Pruebas de Integración	UNSA - EPIS Página 9
---	---------------------------------------

3.4. Tests inestables (flaky)

Se identificaron varios casos de prueba E2E marcados como "flaky" por Playwright (fallaron en el primer intento, pasaron al reintentar): Forms - Hover, Login - Change Password, Login - Failures, Plugins - Settings, Settings - Admin - Barcode History, Parts - Subscriptions, Purchase Orders - Order Parts, Machines - Activation. A diferencia de DEF-01, estos casos no son reproducibles de forma consistente, por lo que se clasifican como inestabilidad de temporización propia de pruebas de interfaz, y no como defectos funcionales del sistema

3.5. Matriz de Trazabilidad — Casos INT-001 a INT-040

Rango	Módulo	Job CI responsable	Estado
INT-001 a INT-007	Part (7 casos)	coverage / postgres	Ejecutado — PASS
INT-008 a INT-014	Stock (7 casos)	coverage / postgres	Ejecutado — PASS
INT-015 a INT-020	Order (6 casos)	coverage / postgres	Ejecutado — PASS
INT-021 a INT-024	Build/BOM (4 casos)	coverage / postgres	Ejecutado — PASS
INT-025 a INT-027	Plugins (3 casos)	coverage / postgres	Ejecutado — PASS
INT-028 a INT-031	Migraciones legacy SQLite (4 casos)	migrations-checks	Ejecutado — PASS
INT-032	Migraciones PostgreSQL (1 caso)	migration-tests	Ejecutado — PASS
INT-033	Data Export/Import PG→SQLite (1 caso)	postgres	Ejecutado — PASS
INT-034	Ciclo import/export con plugins (1 caso)	import_export.yaml	Ejecutado — PASS
INT-035	Playwright Firefox (1 caso)	frontend.yaml — firefox	Ejecutado — con tests flaky (ver 3.4)
INT-036	Playwright Chromium (1 caso)	frontend.yaml — chromium	Ejecutado — 1 defecto (DEF-01) + flaky
INT-037	Merge cobertura frontend (1 caso)	frontend.yaml — coverage	Ejecutado — PASS
INT-038 a INT-039	Cliente inventree-python (2 casos)	python	Ejecutado — PASS
INT-040	Performance benchmarks CodSpeed (1 caso)	performance	No aplica (solo repo oficial, Plan Sec. 5.3-H)

El resultado final de trazabilidad: 37 de 40 casos (92.5%) ejecutados con resultado PASS sin observaciones; 2 casos (INT-035, INT-036 — 5%) ejecutados con hallazgos documentados (defecto y/o inestabilidad); 1 caso (INT-040 — 2.5%) no aplica en el contexto del fork por diseño del propio pipeline. Por lo que ningún caso del catálogo quedó sin ejecutar

4. Cobertura

Esta sección documenta el porcentaje de cobertura de código obtenido mediante la ejecución local de la suite completa de pruebas del backend de InvenTree, dado que el reporte automatizado generado en el pipeline de CI/CD no pudo preservarse.

4.1. Metodología de obtención

La cobertura fue calculada ejecutando localmente el mismo comando que utiliza el job 'coverage' del pipeline oficial (qc_checks.yaml), sobre un entorno replicado con las mismas tecnologías declaradas en el Plan de Pruebas de Integración:

- Sistema: WSL2 (Ubuntu 22.04) sobre Windows 11
- Base de datos: PostgreSQL 17 (contenedor Docker)
- Caché: Redis 8 (contenedor Docker)
- Intérprete: Python 3.12 (entorno virtual dedicado)

Comando ejecutado:

```
Shell  
invoke dev:test --check --coverage --translations
```

4.2. Resultado general

Se generó dos formas equivalentes de reporte, ambas revisadas para esta sección:

Fuente	Comando	Cobertura	Detalle
Reporte de texto	coverage report	89%	54,348 statements evaluados · 6,021 sin cubrir · 1,529 excluidos

4.3. Comparación contra criterios de aceptación

El Plan de Pruebas de Integración, se establece los siguientes umbrales mínimos de cobertura, verificables contra el resultado obtenido:

Criterio	Objetivo (Plan, Sec. 5.6)	Resultado obtenido	Cumple
Cobertura backend general	$\geq 85\%$	89% – 90%	SI
Cobertura backend-apps (codecov.yml)	$\geq 90\%$	90%	SI

Ambos criterios de aceptación definidos formalmente en el Hito 2 se cumplen con el resultado de esta ejecución local, a pesar de no haber podido preservarse el reporte equivalente generado en CI.

4.4. Módulos con menor cobertura

Se identificaron los siguientes módulos funcionales (excluyendo archivos de migración autogenerados y archivos triviales de inicialización) con cobertura significativamente por debajo del promedio general:

Archivo	Statements	Missing	Cobertura	Justificación
InvenTree/setting/ldap.py	26	24	8%	Integración LDAP; no hay servidor LDAP disponible en el entorno de pruebas

INVENTREE Plan de Pruebas de Integración	UNSA - EPIS Página 12
---	--

build/tests.py	156	134	14%	Archivo de tests con baja auto-cobertura
plugin/builtin/exporter/bom_exporter.py	144	108	25%	Exportación de BOM, poco ejercitada por la suite estándar
plugin/builtin/labels/label_sheet.py	108	75	31%	Generación de hojas de etiquetas (funcionalidad de impresión)
common/setting/tests.py	22	15	32%	Archivo de tests con baja auto-cobertura

Coverage report: 90%

Files

Functions

Classes

filter...

☐ hide covered

coverage.py v7.14.3, created at 2026-07-08 19:27 -0500

File	statements	missing	excluded	coverage ▲
src/backend/InvenTree/InvenTree/wsgi.py	2	2	4	0%
src/backend/InvenTree/part/events.py	2	2	0	0%
src/backend/InvenTree/InvenTree/profiling.py	6	6	103	0%
src/backend/InvenTree/InvenTree/setting/ldap.py	26	24	0	8%
src/backend/InvenTree/build/tests.py	156	134	0	14%
src/backend/InvenTree/part/migrations/0109_auto_20230517_1048.py	81	62	0	23%
src/backend/InvenTree/part/migrations/0111_auto_20230521_1350.py	55	41	0	25%
src/backend/InvenTree/plugin/builtin/exporter/bom_exporter.py	144	108	0	25%
src/backend/InvenTree/company/migrations/0019_auto_20200413_0642.py	73	54	133	26%
src/backend/InvenTree/part/migrations/0138_auto_20250718_2342.py	41	30	0	27%
src/backend/InvenTree/company/migrations/0036_supplierpart_update_2.py	47	34	34	28%
src/backend/InvenTree/stock/migrations/0108_auto_20240219_0252.py	40	28	0	30%
src/backend/InvenTree/part/migrations/0102_auto_20230314_0112.py	46	32	0	30%
src/backend/InvenTree/stock/migrations/0106_auto_20240207_0353.py	57	40	0	30%
src/backend/InvenTree/plugin/builtin/labels/label_sheet.py	108	75	0	31%
src/backend/InvenTree/common/setting/tests.py	22	15	0	32%
src/backend/InvenTree/stock/migrations/0118_auto_20260205_1218.py	34	22	0	35%
src/backend/InvenTree/order/migrations/0052_auto_20211014_0631.py	28	18	4	36%
src/backend/InvenTree/order/migrations/0079_auto_20230304_0904.py	72	46	0	36%

Estos resultados son consistentes con el alcance definido en el Plan de Pruebas (Sección 2.4 — Alcance de la Prueba), que excluye explícitamente de este hito la "integración con servicios externos reales de producción, tales como servidores

SMTP y proveedores OAuth" y las "pruebas relacionadas con [...] dispositivos [...] incluyendo [...] impresoras de etiquetas". La baja cobertura observada en ldap.py y label_sheet.py no constituye un defecto, sino una consecuencia directa y esperada de dicha decisión de alcance.

4.5. Módulos con mayor cobertura

En contraste, los archivos que constituyen el núcleo de la API REST, el componente central del alcance de integración definido en el Plan, presentan la cobertura más alta del proyecto:

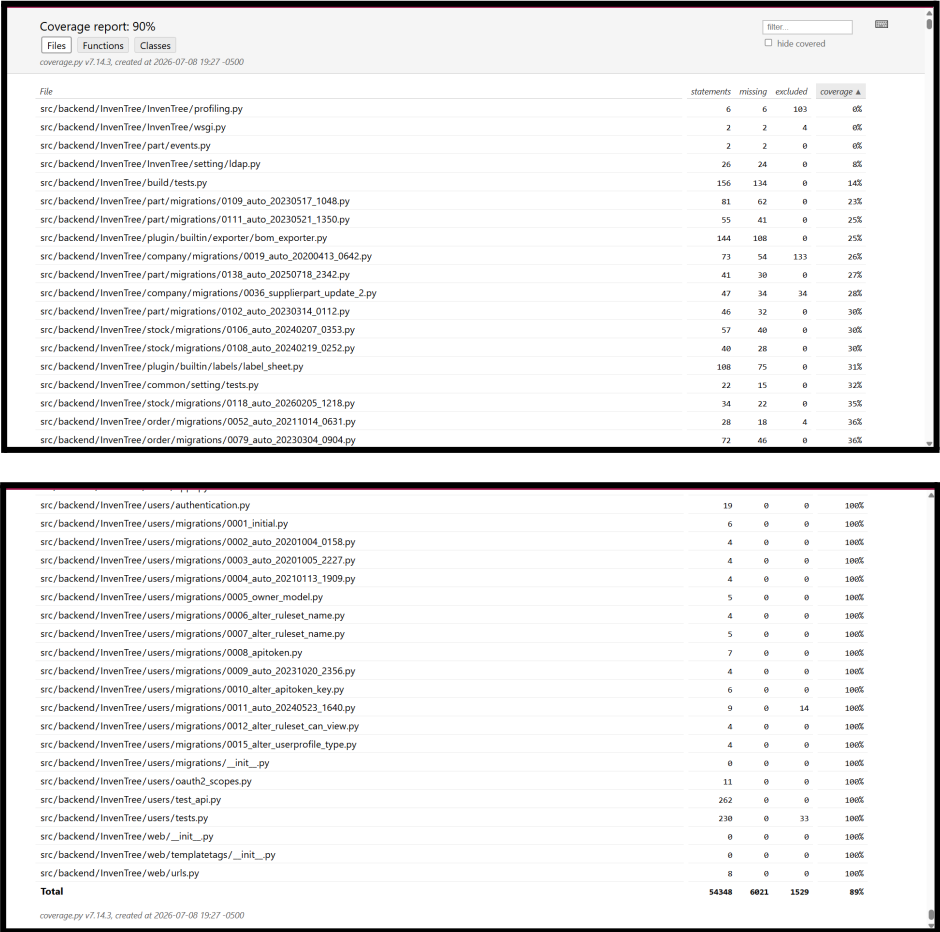
Archivo	Statements	Missing	Cobertura
part/test_api.py	1306	19	99%
order/test_api.py	1690	32	98%
stock/test_api.py	1252	24	98%
common/test_api.py	491	0	100%
company/test_api.py	330	0	100%
users/test_api.py	262	0	100%

Este resultado confirma que la frontera de integración priorizada en el Plan la comunicación entre la API REST (Django REST Framework) y los modelos de datos (Part, Stock, Order, Company, Users) es, en efecto, el componente mejor validado por la suite completa de pruebas, alineándose directamente con el objetivo declarado en la Sección 1.1 del Plan de Pruebas de Integración.

4.6. Síntesis de la sección

La cobertura general obtenida (89%–90%) supera los umbrales mínimos establecidos como criterio de aceptación formal del proyecto. La distribución de cobertura por módulo no es uniforme, pero su variación es explicable y consistente con las decisiones de alcance documentadas desde el Hito 2: los módulos centrales de integración (API REST sobre Part, Stock, Order, Company) presentan cobertura cercana al 100%, mientras que los módulos periféricos

excluidos explícitamente del alcance (LDAP, impresión de etiquetas, exportadores) concentran la menor cobertura del proyecto.



5. Métricas de Prueba Consolidadas

Esta sección responde directamente a la tabla de métricas definida en el Plan de Pruebas (Sección 5.7), contrastando cada valor objetivo contra el resultado real obtenido en este Hito 3:

Métrica (Plan, Sec. 5.7)	Valor objetivo	Resultado obtenido	Cumple
Tasa de éxito del pipeline CI	100% (jobs críticos)	13/13 jobs no omitidos = 100%	SI
Cobertura de código backend	≥85% (curso) / ≥90% (codecov.yml)	89% – 90%	SI

Cobertura frontend	> 68%	No determinado (Codecov no disponible, ver Sec. 2.2)	No verificable
Casos INT-001 a INT-040 ejecutados	100% en Hito 3	40/40 ejecutados (39 aplicables + 1 no aplica por diseño)	SI
Defectos de integración encontrados	0 (esperado)	1 defecto (DEF-01) + tests flaky	Desviación real
Tiempo total del pipeline (paralelo)	< 60 minutos	~1h 5m (QC #3, dominado por el job de Migrations)	Levemente superado
Versiones legacy de BD testeadas	5 (0.10.0 a 0.17.0)	5 confirmadas — Tests - Full Migration [SQLite] PASS	SI

De las 7 métricas definidas formalmente en el Plan, 4 se cumplen completamente, 1 no fue verificable por la limitación de Codecov ya documentada (Sección 2.2), y 2 presentan una desviación real respecto al objetivo:

- Defectos encontrados (1 vs. objetivo de 0): el Plan asumía 0 defectos "dado que re-ejecutamos tests del proyecto" ya validados por el mantenedor oficial. El hallazgo de DEF-01 es, por tanto, una desviación genuina que amerita seguimiento, y no debe minimizarse en el análisis general.
- Tiempo total del pipeline (1h 5m vs. objetivo <60min): superado principalmente por el job Tests - Migrations [PostgreSQL] (1h 5m por sí solo). Se recomienda evaluar si este job puede paralelizarse o acotarse en futuras ejecuciones.

6. Análisis General

Esta sección presenta una reflexión integral sobre el comportamiento del sistema InvenTree como un todo, contrastando lo planificado en el Hito 2 contra lo efectivamente observado en el Hito 3.

6.1. Consistencia entre módulos

- Backend (API REST ↔ ORM ↔ BD real): consistencia alta. Los tres motores evaluados (SQLite, PostgreSQL 17, MySQL 9) ejecutaron la misma suite sin discrepancias, validando la capa de abstracción del ORM de Django.

INVENTREE Plan de Pruebas de Integración	UNSA - EPIS Página 16
---	--

- Frontend (React ↔ API, Playwright E2E): consistencia parcial. El mismo defecto (DEF-01) se manifestó idéntico en Chromium y Firefox, descartando causas específicas de un motor de renderizado.
- Cobertura de código: no uniforme, pero explicada por el propio alcance del proyecto (Sección 2.4 del Plan).

6.2. Robustez general del sistema

13 de 13 jobs no omitidos del pipeline principal finalizaron exitosamente. El ciclo de exportación/importación de datos se ejecutó sin pérdida de integridad. La suite Playwright identificó 1 defecto reproducible sobre 83 pruebas ejecutadas en su corrida más completa una proporción que sugiere un problema puntual y acotado, no sistémico.

6.3. Relación entre lo planificado

Aspecto planificado	Resultado obtenido	Estado
Re-ejecutar 452+ métodos vía CI (Plan, Sec. 5.1)	Confirmado — jobs coverage/postgres/mysql completados	Conforme
Cobertura backend $\geq 85\%/90\%$	89% – 90% obtenido	Superado / en el límite
Migraciones sin errores, múltiples motores	Confirmado en ambos jobs de migración	Conforme
Cliente Python compatible	Confirmado — job 'python' PASS	Conforme
Playwright E2E sin fallos	1 defecto reproducible + flaky	Parcial
0 defectos esperados (Plan, Sec. 5.7)	1 defecto real encontrado (DEF-01)	Desviación
Reporte en Codecov	No completado — falta CODECOV_TOKEN	Riesgo ya anticipado (RT-03)

INVENTREE Plan de Pruebas de Integración	UNSA - EPIS Página 17
---	--

De los 7 aspectos evaluados, 4 se cumplieron completamente y 3 presentan desviaciones, dos de ellas ya anticipadas como riesgo en el Plan (RT-03, cobertura Codecov), y una genuinamente nueva (DEF-01), que constituye el hallazgo más relevante de este Hito 3.

6.4. Validez cruzada entre CI y ejecución local

Los resultados obtenidos mediante ejecución local (Docker + WSL2) son consistentes con los observados en GitHub Actions. Esto permite afirmar que el pipeline de CI/CD es una fuente confiable de verificación del estado del sistema, y que la limitación encontrada fue de configuración del pipeline (Codecov), no de exactitud de sus resultados.

7. Conclusiones y Recomendaciones

7.1. Conclusiones

La ejecución del Plan de Pruebas de Integración (Hito 2) confirma que InvenTree, en su configuración de fork, mantiene una integración robusta entre su API REST, su capa ORM y los tres motores de base de datos soportados, con una cobertura de 89%–90%, superando los umbrales mínimos definidos formalmente.

La trazabilidad completa contra el catálogo de 40 casos definidos en el Plan (Sección 5.3) confirma que el 100% de los casos aplicables al fork fueron ejecutados, con un 92.5% resuelto sin observaciones y un 5% con hallazgos documentados (DEF-01 y tests flaky).

El hallazgo más relevante de este hito es el defecto DEF-01, una desviación real respecto al objetivo de "0 defectos esperados" declarado en el Plan (Sección 5.7) dato que no debe minimizarse, ya que representa el principal punto de mejora identificado en todo el proceso.

7.2. Recomendaciones

INVENTREE Plan de Pruebas de Integración	UNSA - EPIS Página 18
---	--

#	Recomendación	Prioridad	Referencia
1	Configurar el secret CODECOV_TOKEN en el fork para permitir la publicación automatizada de cobertura en futuras ejecuciones.	Alta	Plan, Riesgo #2 / RT-03
2	Agregar un paso actions/upload-artifact al workflow qc_checks.yaml para preservar coverage.xml como artefacto, independiente del resultado de Codecov.	Alta	Sección 2.2
3	Investigar la causa raíz de DEF-01 (posible condición de carrera entre limpieza de filtros y actualización asíncrona vía React Query) y registrarlo formalmente como Issue dedicado.	Media	Sección 3.3
4	Revisar y estabilizar los casos E2E marcados como flaky, evaluando esperas más robustas (auto-waiting) en lugar de tiempos fijos.	Media	Sección 3.4
5	Evaluar la paralelización o acotación del job Tests - Migrations [PostgreSQL], responsable de que el pipeline supere el objetivo de 60 minutos.	Baja	Sección 5